Design note: Profile Builder

Aniket Mishra · AI Engineering take-home · FastAPI, PostgreSQL, React, Gemini. Diagrams: `docs/diagrams/architecture.png` and `data-model.png`. One decision record per major choice: `docs/decisions/`.

What it does. After a task closes, the admin panel calls `POST /v1/sessions`. An AI-led debrief of 3 to 7 questions runs with the human agent (typed or voice). The AI proposes profile facts, each quoting the agent's own words. The agent keeps, edits or discards them, and they are saved into an append-only, per-user profile that other AI systems read as JSON, history, or an LLM-ready text block. No LLM runs on the read path.

1. LLM and prompting strategy

- **Three small calls, not one open chat.** Plan the questions (once per session, started in the background when the session is created, 8 s timeout). Interpret each answer (clear or ambiguous, which later questions are already covered; 4 s timeout). Extract facts (once, when the debrief ends; 20 s timeout). I chose this so each call is easy to test, time-box and replace.
- **Code owns flow control, the LLM only advises.** The question cap (7, follow-ups included), one follow-up per question, skip, end early and stale-answer rejection (409) are in `debrief.py`. Every model output is validated; made-up question labels are ignored.
- **Prompts are versioned files** (`planner_v1.md`, `interpreter_v1.md`, `extractor_v1.md`). The file name is stored with results, so a fact can be traced to its prompt version and model.
- **One provider interface.** A provider implements `complete_json(system, user)`. `generate_structured` validates the reply against a Pydantic schema and retries once with the validation errors shown, then raises. The schema is placed in the prompt text, so swapping providers does not depend on any vendor's schema dialect. The trade-off: JSON mode with a prompt schema is less constrained than native tool calling.
- **A safe fallback at every step.** No LLM or a failure gives task-aware template questions, an answer treated as clear, and the agent's own words offered as `notes` facts. Nothing is lost and nothing is invented.
- **Customer text is untrusted.** The transcript excerpt and agent answers are wrapped in tags, closing tags are neutralised, and the prompts say to treat them as data. Questions may only refer to what the excerpt literally says.

2. Extraction schema

Each saved fact has `category`, `key`, `value` (string or list of strings), `confidence` (0 to 1), `evidence`, `source_session_id`, `captured_at`, plus `is_current`, `superseded_by` and `provenance` (prompt version, model, attempt, whether the agent edited it). Seven categories: `contact_preferences`, `communication_style`, `products_and_usage`, `constraints`, `pain_points`, `personal_context`, `notes`. Keys are snake_case; the prompt lists canonical keys but new ones are allowed.

Validation is code, not trust (`extraction.sanitize`). A proposal is dropped, with the reason stored in `extractions.dropped`, if its key is malformed, its value is empty, its confidence is below 0.5, it repeats a (category, key), it exceeds 20 facts, or its **evidence quote does not appear inside a single agent answer** (case, punctuation and whitespace ignored; questions and task context do not count). Confidence follows a rubric in the prompt (0.9+ stated directly, 0.7 to 0.89 hedged, 0.5 to 0.69 implied). It is the model's own estimate and is not calibrated. An agent edit sets it to 1.0.

3. Conflict rule (v1)

The latest confirmed write for a key wins. The old row stays with `is_current = false`, `superseded_by` and `superseded_at`, and keeps its source session. The same value again is a no-op. Inside one batch, duplicate keys keep the highest confidence (later wins ties). Confidence is not used across sessions, because an old confident guess should not beat a newer fact the agent confirmed. A partial unique index allows exactly one current row per (tenant, user, category, key). Writers are serialised per profile with a row lock on `end_users`, and `profile_version` increases on each change. Only `services.apply_facts()` writes facts, and the confirm step saves facts and the extraction status in one transaction.

4. How isolation is enforced

1. **One place.** `repo.py` is the only module that queries tenant data. `ScopedRepo` is built from the tenant resolved from the API key, no method takes a tenant argument, and every query goes through `_select()`, which adds the tenant filter. The one query that runs before a tenant is known is the API key lookup.
2. **The database backs it up.** Every foreign key between tenant tables is composite `(tenant_id, id)`, so a fact cannot reference another tenant's user or session.
3. **The chat token is narrow.** It is signed, expires, and is bound to one session; its tenant comes from the signed claims. It reaches only that session's debrief, voice and extraction routes, never profiles.
4. **Other tenants see 404, not 403**, so existence does not leak. The body `tenant_id` on intake must match the key (403 otherwise).

Tests (`test_isolation.py`, plus cross-tenant cases in the debrief, extraction and voice tests) cover reads, writes, history, context, sessions, debriefs, extraction, voice and a revoked key. Both SQLite and Postgres must pass.

5. Async work

Plan precompute, extraction and transcription are in-process background tasks whose state is saved in the database (`debriefs`, `extractions`, `voice_notes`). A crash marks a run `failed` or leaves it `running`; a `running` extraction older than 120 s can be re-run, and a stuck voice note is re-queued by the next poll (3 attempts). Trade-off accepted: no separate worker, so a restart delays a job until the next request touches it instead of resuming it by itself.

6. Scaling: what breaks first

This is analysis, not measurement. No load test was run, and the latency targets (first question about 3 s, turn about 5 s, profile read under 200 ms) have not been measured on the hosted app.

1. **LLM calls inside request transactions.** `debrief.submit` locks the debrief row and calls the interpreter before it commits, so a database connection is held for up to the 4 s timeout, longer if the provider retries. The pool is 5 connections plus 5 overflow per process. If a fifth of 200 concurrent sessions submit in the same few seconds, about 40 requests compete for 10 connections. Fix: read state, release the connection, call the model, then re-lock and re-check the question id; add PgBouncer.
2. **Provider rate limits.** There is one retry and no queue or limiter. Fallbacks keep the debrief working but with plainer questions. Fix: a job queue with a concurrency limit and backoff, and a paid tier.
3. **In-process jobs** do not survive restarts on their own and do not spread across instances. Fix: a jobs table and a worker using `SELECT ... FOR UPDATE SKIP LOCKED`.
4. **Reads (50 per second).** Each read does three indexed queries (API key by prefix, end user, current facts). That should be comfortable on Postgres, but I would add a short key-to-tenant cache, an ETag from `profile_version`, and a read replica. History is not paginated.
5. **Data growth (1M profiles).** `extractions.llm_input` stores the user prompt, which contains the agent's answers, and chat messages hold them too, so a retention job is needed. Audio is deleted after transcription, which limits bloat.

7. Limitations

- Not built: the `profile.updated` webhook, the dashboard, the PII redaction toggle, a separate worker with a jobs table, and the demo video.
- The LLM and speech providers were tested with scripted outputs, HTTP mocks and a stub. Real question quality, extraction accuracy and mixed Hindi and English speech are unmeasured, and there is no evaluation set.
- The UI was tested with component tests and a simulated browser, not in a real browser. The microphone path is untested on real devices.
- Confidence is not calibrated. The evidence check proves a quote exists, not that the value is the right reading; the agent's review is the backstop.
- Multi-valued keys overwrite unless the value is a list. There is no rate limiting, key rotation, token revocation or transcript retention policy.
- The demo deployment enables debug endpoints (hand-written facts) and `frame-ancestors *`. The mock admin panel holds the API key in the browser. Both are demo shortcuts and must change before real use.

8. With another week

1. Run a real evaluation: a golden set of debriefs, extraction precision, and edit rate per prompt version.
2. Move model calls out of transactions, then add the jobs table and worker.
3. `profile.updated` webhook (the `profile_version` counter is the hook), then the dashboard from the `agent_edited` and voice `edited` signals already stored.
4. PII redaction toggle on `/context`, Postgres row-level security as a second isolation layer, per-key cardinality, rate limiting and key rotation.